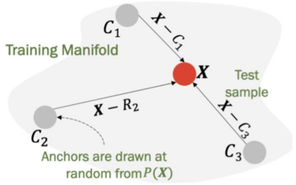
Accurate And Scalable Estimation Of Epistemic Uncertainty For Graph Neural Networks

Puja Trivedi[1], Mark Heimann[2], Rushil Anirudh[2], Danai Koutra[1] Jayaraman J. Thiagarajan[2]

Stochastic Centering: Single Model Ensembles

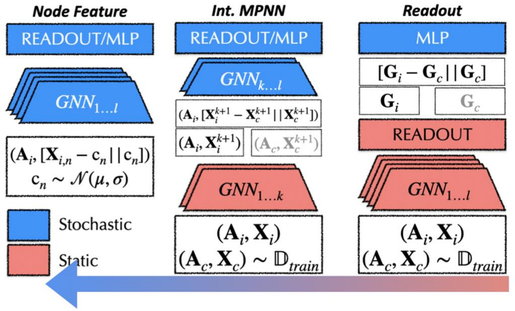
NTKs for (Graph) Neural Networks are not shift invariant → The variability over shifts captures epistemic uncertainty!

- Stochastic Centering creates a ***relative representation*** for an input, ***x***, in terms of a random ***anchor***, ***c***.
- During training, the anchor is ***randomized,*** thus implicitly sampling ***different hypotheses.***



- During testing, predictions are computed over ***multiple*** anchors.
- ***Discrepancy*** across anchors choices captures **epistemic uncertainty**!

G Δ-UQ: Stochastic Centering for GNNs



Anchored Inference

Prediction:

$$\boldsymbol{\mu}(y|\mathcal{G}_i) = \frac{1}{K}\sum_{k=1}^{K} f_\theta([\mathcal{G}_i, \mathbf{c}_k])$$

Uncertainty:

$$\boldsymbol{\sigma}(y|\mathcal{G}_i) = \sqrt{\frac{1}{K-1}\sum_{k=1}^{K}(f_\theta([\mathcal{G}_i, \mathbf{c}_k]) - \boldsymbol{\mu})^2}$$

Calibrated Prediction:

$$\boldsymbol{\mu}_{calib} = \boldsymbol{\mu}(1 - \boldsymbol{\sigma})$$

- Since graphs are variable sized and discrete, we introduce three anchoring strategies tailored for GNNs.
- Induce **varying levels of stochasticity**, trading off hypothesis diversity and the semantic diversity of the anchoring distribution.

We propose G Δ-UQ, a novel training paradigm for improving calibration with GNNs

- ✓ Scalable and Lightweight
- ✓ Robust to Distribution Shifts
- ✓ Supports Pretrained Models
- ✓ Easy to Implement

```python
class GNN_ReadOutAnc:
    def __init__(GNN):
        self.MPNN1 = MPNN(InputDim,HDim1)
        self.MPNN2 = MPNN(HDim1, HDim2)
        self.MPNN3 = MPNN(HDim2, HDim3)
        self.READOUT = READOUTLayer()
        #Corres. to GDUQLayer!
        self.classifier = Linear(HDim3 * 2, NumClass)

    def forward(X,Adj):
        h = self.MPNN1(X,Adj)
        h = self.MPNN2(h,Adj)
        h = self.MPNN3(h,Adj)
        g = self.READOUT(h,Batch) #OUT: (B, HDim3)
        C = SHUFFLE(g, dim=0) #(B, HDim3)
        gAnc = CONCAT([g-C,C],dim=1)#(B, HDim3 x 2)
        pred = self.classifier(gAnc) #Out: (B, NumClass)
        return pred
```

G Δ-UQ is easy to implement, with only a few changes to standard GNNs !

```python
#training
for (G,y) in trainloader:
    C = create_anchors(n = Batch$Anc)= CONCAT([G-C,C], dim = 1)
    preds = model(G_Anc)
    loss = criterion(preds,Y)

#inference
anc_preds = []
testAncs = create_anchors(n = K)
for A in testAncs:
    G_anc = CONCAT([I-A,A], dim = 1)
    preds = model(G_anc)
    anc_preds.append(preds)
P = CONCAT(anc_preds, dim = 0)
mu = MEAN(P, dim = 0)
var = STDDEV(P, dim = 0)
```

Improved Node Classification Calibration

We evaluate G Δ-UQ with node feature anchoring on:

- 4 Datasets
- 2 Distribution shifts
- 8 *Post Hoc* Calibration

G Δ-UQ matches or surpasses accuracy and calibration of the vanilla GNN on 8/8 datasets.

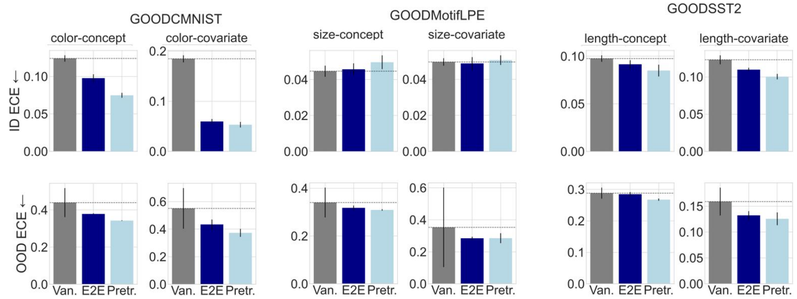
Dataset	Domain	Calibration	Shift: Concept						Shift: Covariate					
			Accuracy (↑)			ECE (↓)			Accuracy (↑)			ECE (↓)		
			No G-Δ UQ	G-Δ UQ		No G-Δ UQ	G-Δ UQ		No G-Δ UQ	G-Δ UQ		No G-Δ UQ	G-Δ UQ	
WebKB	University	×	0.253±0.003	0.281±0.009		0.67±0.061	0.593±0.025		0.122±0.029	0.115±0.041		0.599±0.091	0.525±0.033	
		CAGCN	0.253±0.005	0.268±0.008		0.452±0.14	0.473±0.12		0.122±0.018	0.092±0.161		0.355±0.227	0.396±0.161	
		Dirichlet	0.229±0.016	0.22±0.022		0.472±0.06	0.472±0.03		0.244±0.105	0.295±0.044		0.299±0.092	0.329±0.044	
		ETS	0.253±0.005	0.273±0.012		0.64±0.06	0.575±0.019		0.121±0.021	0.084±0.037		0.539±0.112	0.499±0.027	
		Ord	[illegible]	[illegible]		[illegible]	[illegible]		[illegible]	[illegible]		[illegible]	[illegible]	
Cora	Degree	[illegible]	[illegible]	[illegible]		[illegible]	[illegible]		[illegible]	[illegible]		[illegible]	[illegible]	
		Ord	[illegible]	[illegible]		[illegible]	[illegible]		[illegible]	[illegible]		[illegible]	[illegible]	
Cora	Word	[illegible]	[illegible]	[illegible]		[illegible]	[illegible]		[illegible]	[illegible]		[illegible]	[illegible]	
		Ord	[illegible]	[illegible]		[illegible]	[illegible]		[illegible]	[illegible]		[illegible]	[illegible]	
		VS	0.607±0.001	1.683±0.002		0.285±0.005	0.111±0.003		0.083±0.004	0.636±0.002		0.261±0.005	0.313±0.003	
CBAS	Color	×	0.83±0.014	0.829±0.011		0.169±0.013	0.151±0.014		0.703±0.015	0.746±0.027		0.266±0.02	0.169±0.018	
		CAGCN	0.83±0.013	0.83±0.013		0.137±0.011	0.143±0.022		0.703±0.019	0.749±0.033		0.25±0.021	0.198±0.017	
		Dirichlet	0.801±0.02	0.806±0.008		0.161±0.012	0.17±0.01		0.671±0.018	0.771±0.03		0.241±0.029	0.217±0.017	
		ETS	0.83±0.013	0.827±0.014		0.146±0.013	0.164±0.007		0.703±0.019	0.76±0.037		0.28±0.023	0.176±0.019	
		GATS	0.83±0.013	0.83±0.021		0.16±0.009	0.173±0.021		0.703±0.019	0.751±0.016		0.236±0.039	0.16±0.015	
		IRM	0.829±0.013	0.829±0.015		0.142±0.009	0.133±0.006		0.72±0.019	0.803±0.04		0.207±0.035	0.158±0.017	
		Orderinvariant	0.83±0.013	0.803±0.008		0.174±0.006	0.173±0.009		0.703±0.019	0.766±0.045		0.261±0.017	0.194±0.031	
		Spline	0.82±0.016	0.824±0.011		0.159±0.009	0.16±0.014		0.683±0.019	0.786±0.038		0.225±0.034	0.179±0.035	
		VS	0.829±0.012	0.840±0.011		0.196±0.011	0.146±0.012		0.717±0.019	0.809±0.008		0.242±0.019	0.182±0.014	

G Δ-UQ + Post Hoc. Calib achieves:

- **Best** accuracy on 8/8 datasets
- **Best** or **2nd best** calibration on 8/8 datasets

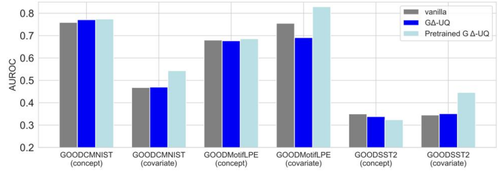
Better Graph Classification Calibration

G Δ-UQ Improves Calibration of Pretrained GNNs

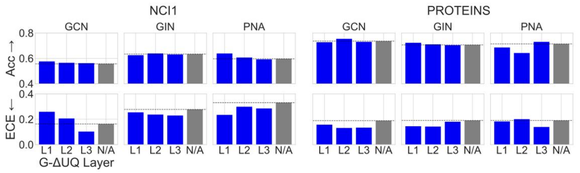


G Δ-UQ Estimates are Effective in Safety Critical Tasks

- G Δ-UQ achieves the best OOD detection AUROC on 5/6 datasets.
- G Δ-UQ also performs well on the generalization gap prediction tasks.



G Δ-UQ Improves Graph Classification Calibration with Different MPNNs



- G Δ-UQ is flexible to message passing layer.
- Required level of stochasticity may vary for dataset/GNN.

Check out the G Δ-UQ Family!

Try E Δ-UQ for link prediction calibration!